LLM Vulnerability Detection on the CASTLE Benchmark: Effects of Prompting and Temperature

Kamil Jabłkowski
The Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
kamil.jablkowski.stud@pw.edu.pl

Paweł Borkowski
The Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
pawel.borkowski4.stud@pw.edu.pl

Bartosz Sawicki
The Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
bartosz.sawicki@pw.edu.pl

***Abstract*—The increasing complexity and scale of modern software systems, combined with the rising number of reported security vulnerabilities, have intensified the need for effective automated vulnerability detection. Large language models (LLMs) have demonstrated their potential for code analysis tasks, yet their reliability in security tasks remains underexplored. This paper evaluates LLM-based vulnerability detection and the impact of prompting and generation settings. We evaluated four LLMs on the CASTLE Benchmark using three prompting strategies: zero-shot, role-based and chain-of-thought. Performance was measured with accuracy, precision, recall, F1 score, and processing cost. The results indicate that LLMs can effectively support vulnerability detection, with GPT-5.1 performing best. Our study shows that these models can potentially complement traditional static analysis tools in modern software security workflows.**

***Index Terms*—Vulnerability Detection, Large Language Models, Prompt Engineering, CWE**

I. Introduction

In today's world, there is a strong trend toward IT transformation and the digitalization of all types of services [1], [2]. Rapid changes in this area also introduce new challenges, including increased exposure to cyberattacks [3]. As reported by Ze Sheng et al., 70% of security vulnerabilities stem from human errors made during software development [4], motivating early vulnerability detection tools.

Traditional tools such as static application security testing (SAST) still offer insufficient detection accuracy. The study by Goseva-Popstojanova and Perhinschi shows that many popular SAST tools achieve recall levels often below 50% [5]. At the same time, these tools exhibit large numbers of both false negatives and false positives.

Recently, large language models (LLMs) have gained substantial popularity. LLMs can understand code semantically and generate correct program fragments – a capability demonstrated by Giuseppe Destefanis et al., even for the relatively outdated GPT-3.5 model [6]. Preliminary research also suggests that LLMs may detect bugs missed by traditional analyzers, but their vulnerability-detection performance remains inconsistent and highly dependent on prompt formulation and input context. Prompting strategy may also affect effectiveness in security-related tasks [7].

Realistic benchmarks are limited as many datasets contain short synthetic snippets. We therefore use the CASTLE Benchmark [8], which provides 250 hand-crafted C programs covering 25 CWE categories. This allows a far more credible and nuanced evaluation of security tools, including LLMs.

This study evaluates selected LLMs on the CASTLE dataset and examines the effects of prompting strategies and temperature. We address the following research questions:

- **RQ1**: How effective are LLMs at vulnerability detection?
- **RQ2**: How does prompting affect performance?
- **RQ3**: How does temperature affect performance?

Our study holds the following contributions:

- Evaluation of LLM detection performance across top 25 Common Weakness Enumeration (CWE) categories.
- Comparison of zero-shot, role-based and chain-of-thought prompting strategies.
- Analysis of temperature effects on model performance.

II. Methodology

This section outlines the experimental setup for assessing vulnerability detection in selected LLMs.

A. Research Design

We evaluated LLM-based vulnerability detection in C programs under different prompts and temperature settings. Each sample was tested independently.

B. Dataset

For the experimental evaluation, we used the CASTLE Benchmark dataset, available at: `https://github.com/CASTLE-Benchmark`.

The dataset contains 250 C programs: 150 vulnerable (with at least one documented security vulnerability) and 100 non-vulnerable samples to assess the model's precision and false positive rate.

Each C source file was treated as an independent evaluation instance and submitted separately to the OpenRouter API.

C. Implementation and Experimental Infrastructure

To automate the evaluation process, we developed a Python-based script that handled data processing, prompt generation, API communication, and response parsing.

Models were accessed through the OpenRouter API, which provided unified access to multiple LLM providers. Each

request contained one C file embedded within a structured prompt. The system ensured that every file was evaluated independently to avoid cross-sample information leakage.

D. Evaluated Models

The following large language models were included in the study:

- `grok-code-fast-1`
- `claude-sonnet-4.5`
- `gpt-5.1`
- `devstral-2512`

These models were selected to represent different architectural families and training paradigms, including models optimized for code understanding and general-purpose LLMs.

E. Prompting Strategies

We evaluated three prompting strategies.

1) Zero-shot Prompting: This baseline strategy employed direct vulnerability classification with a strictly formatted JSON output:

```
Examine the following code snippet and
    determine whether it contains a
security vulnerability.

Return ONLY a JSON object in the following
    format, and nothing else:

{
  "vulnerable": true or false,
  "cwe": CODE_NUMBER
}

Code snippet: ...
```

2) Role-Based Prompting: In this strategy, the model was assigned an expert role to potentially enhance analytical rigour and prime its reasoning toward a stronger security focus:

```
You are an expert security analyst
    specializing in vulnerability detection in
C/C++ applications. Review the following code
    snippet to identify potential
vulnerabilities.

Return ONLY a JSON object...
```

3) Chain-of-Thought Prompting: The third strategy encouraged structured internal reasoning while explicitly suppressing explanation in the final output:

```
Examine the following code snippet and
    determine whether it contains a
security vulnerability. Think step by step
    internally. Do not include your
reasoning in the output.

Return ONLY a JSON object...
```

This approach was designed to evaluate whether implicit step-by-step reasoning improves classification accuracy without exposing intermediate reasoning.

F. Output Format and Parsing

All models were instructed to return strictly formatted JSON responses containing:

- a boolean field `"vulnerable"` indicating whether the code contains a vulnerability,
- a numeric `"cwe"` identifier corresponding to the detected Common Weakness Enumeration (CWE) category.

Responses were automatically parsed and validated. Invalid JSON outputs were discarded.

III. Results

We compared LLM performance in vulnerability detection. The bar chart (Fig. 1) illustrates F1 scores for several LLMs using different prompting strategies: zero-shot, role-based, and chain-of-thought. Most models score roughly between 0.57 and 0.65 in terms of the F1 score, indicating moderate but not yet satisfactory performance, especially considering that the best possible value of the F1 score is 1.0. The best results come from GPT-5.1, which reaches about 0.65 in the zero-shot strategy and slightly less with role-based prompts. In contrast, Devstral 2 2512 performs the worst, remaining at roughly 0.40 regardless of the prompting strategy employed. Grok Code Fast 1 and Claude Sonnet 4.5 lie in the middle range, showing stable but not outstanding results. Differences between prompting strategies are relatively small and may fall within the bounds of experimental variability.

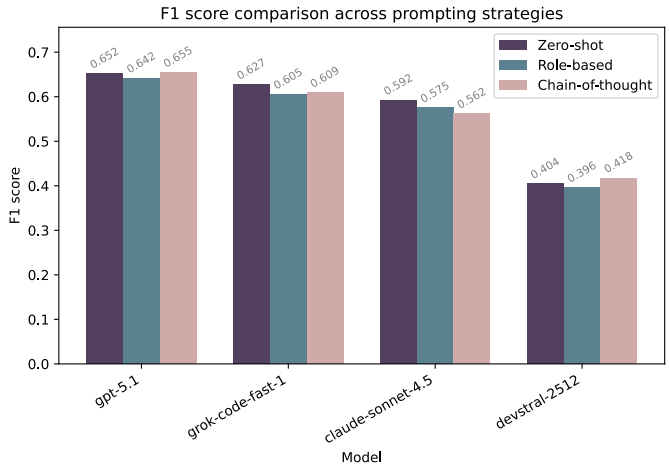

Fig. 1. Prompting strategies comparison

However, it cannot be stated with complete certainty that the type of prompting does not affect the results, as only three basic strategies were tested on four models, which narrows the scope of our conclusions and does not cover more advanced or hybrid techniques. When comparing the prompting strategies directly, chain-of-thought prompts achieved the highest F1 score in exactly two cases, while in the remaining two cases, the best results were obtained with the simpler zero-shot prompting.

Another important part of the research was to examine how changing the temperature – which controls randomness

– influences accuracy. The chart (Fig. 2) illustrates this by comparing the F1 scores of two LLMs evaluated in a zero-shot setup across different temperature values. As we can see, GPT-5.1 performs better than Devstral 2 2512 at every temperature. Its highest F1 score is 0.652 at a temperature of 1.0, with slightly lower values at 0.5 and 1.5. Devstral 2 2512 scores much lower and reacted more strongly to temperature changes: it drops from 0.409 at 0.5 and 0.404 at 1.0 to exactly 0.353 at 1.5. Both models showed a similar trend: a very high temperature (1.5) negatively affected performance. Higher temperature reduces reliability due to increased randomness. One important limitation should be noted: only two models and three temperature values were tested. Because of this, it's hard to know whether the differences are meaningful or just normal experimental variation.

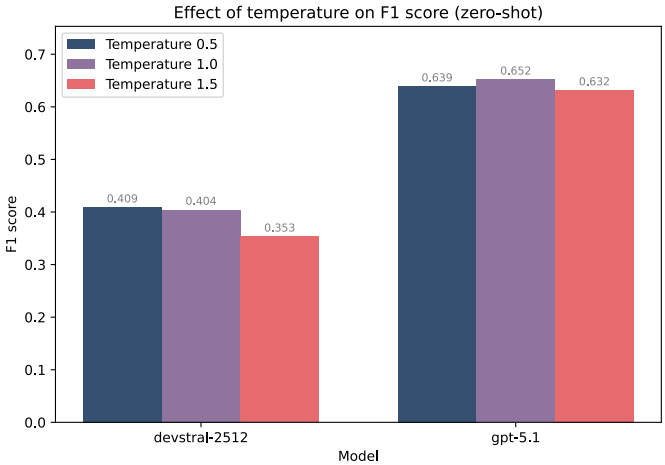

Fig. 2. Temperature comparison

The relationship between the average cost per sample and model performance, measured by F1 score, for LLMs evaluated on vulnerability detection is illustrated below (Fig. 3). The results reveal a clear trade-off between cost and performance: higher-cost models consistently achieve better detection performance, while lower-cost models perform worse. Importantly, no model achieved high effectiveness at a low cost, as evidenced by the lack of data points significantly above the performance-cost curve. Similarly, no model was uniquely inefficient, which would be indicated by points falling markedly below the curve.

The total number of incorrectly formatted responses produced by each evaluated LLM was aggregated across all prompting techniques and included the CWE verification stage. An incorrectly formatted response is defined as either a missing output or a response that does not comply with the required JSON schema. This directly reflects how reliably a model follows strict prompt constraints. The results show that GPT-5.1 achieved perfect compliance, producing no incorrectly formatted responses, which indicates a high level of robustness and consistency in following the given output format. In contrast, Grok Code Fast 1 achieved the worst

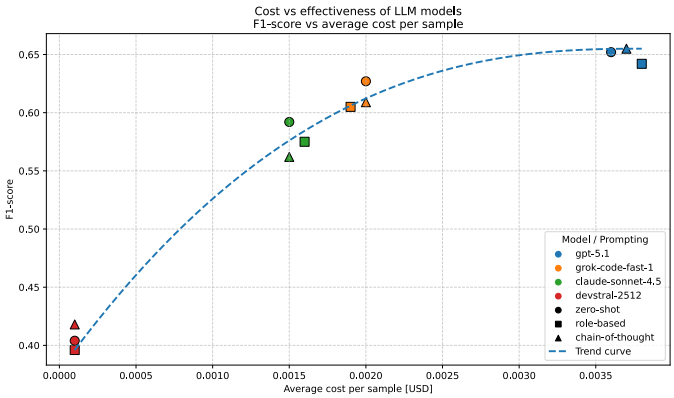

Fig. 3. Cost vs. F1 Score

results, with 33 malformed or missing responses. This was followed by Claude Sonnet 4.5, which produced 29 such cases, while Devstral 2 2512 performed better among the lower-performing models, with 16 incorrect responses. Overall, these findings highlight that LLMs differ in their ability to follow strict output specifications, which is a critical practical aspect for automated vulnerability detection systems.

In the conducted experiments, a prediction was considered successful only if the model detected the presence of a vulnerability and also correctly identified its CWE label according to the ground truth.

IV. Discussion

Our study investigated the ability of LLMs to detect software vulnerabilities. To achieve this, we used the CASTLE Benchmark, which includes hand-crafted C programs and precise CWE labels. We assume the dataset is representative and that the risk of data leakage or benchmark memorization is low, as the dataset is relatively new. We also assumed that using a unified API layer (OpenRouter) would not influence the results, as all experiments were conducted with identical prompt structures, temperature values, and evaluation scripts. While these assumptions simplified the experiments and ensured comparability, the results may still have been affected if some models had prior exposure to similar benchmark-style data. Future work could explore additional datasets and alternative API implementations to validate the generalizability of our findings.

Overall, the obtained results confirmed our expectations. GPT-5.1 consistently outperformed the other models and demonstrated stable behavior across temperature changes. Grok Code Fast 1 and Claude Sonnet 4.5 achieved stable but average results, whereas Devstral 2 2512 performed significantly worse and appeared more sensitive to sampling parameters, despite being the least expensive model. Surprisingly, the zero-shot prompting method yielded slightly better results than both role-based and chain-of-thought approaches, suggesting that additional reasoning instructions do not necessarily yield better outcomes in certain contexts. Similar observations have

been reported by O. Genc [9], where simpler prompting strategies were found to outperform more elaborate prompt designs.

A. Comparison with prior work

TABLE I
COMPARISON OF VULNERABILITY DETECTION PERFORMANCE (CASTLE VS. OUR STUDY)

Model	F1	Precision	Accuracy	Recall
CASTLE Benchmark				
GPT-o3 Mini	0.70	0.63	0.63	0.78
ChatGPT-o1	0.67	0.59	0.60	0.78
DeepSeek R1	0.62	0.47	0.51	0.90
Our Study				
zero-shot / GPT-5.1	0.65	0.72	0.62	0.60
chain-of-thought / GPT-5.1	0.66	0.72	0.62	0.60
role-based / GPT-5.1	0.64	0.66	0.58	0.62

The results obtained in this study were compared with those reported by the authors of the CASTLE Benchmark [8] (Table I). In their evaluation, the best-performing models, such as GPT-o3 Mini, ChatGPT-o1, and DeepSeek R1, achieved F1 scores in the range of 0.62–0.70. In contrast, our best-performing configurations of GPT-5.1 achieved slightly lower F1 scores, ranging from 0.64 to 0.66, depending on the prompting strategy.

In terms of F1 score, our results are broadly comparable to those reported in the original study. The highest F1 score in our experiments (0.66 for the chain-of-thought strategy) is close to the top-performing models from the CASTLE paper, although still slightly below their best reported value of 0.70. This suggests that, despite differences in models and experimental setup, the overall difficulty of the task remains consistent.

Another notable difference lies in precision and recall trade-offs. The models evaluated in the original study tend to achieve higher recall (up to 0.90 in the case of DeepSeek R1), while our GPT-5.1 configurations exhibit higher precision (up to 0.72) but lower recall (around 0.60). This indicates that our models are more conservative, producing fewer false positives at the cost of missing some vulnerabilities.

Overall, the comparison shows that our findings are largely consistent with the CASTLE Benchmark results in terms of relative performance trends, but differences in absolute scores highlight the sensitivity of the evaluation to experimental setup and metric calculation details. This reinforces the importance of standardized evaluation procedures when comparing LLM-based vulnerability detection approaches.

V. CONCLUSION

Our results show that LLMs can effectively support the vulnerability detection task and may serve as an additional security analysis layer. Regarding RQ1, the evaluated models achieved F1 scores broadly comparable to those reported in the original CASTLE Benchmark study, with GPT-5.1 reaching up to 0.66, close to the top-performing models in the literature. This confirms that LLMs may serve as a viable alternative or complement to traditional SAST tools, offering competitive detection performance while remaining widely accessible and easy to integrate. The findings also reveal a clear trade-off between detection quality and cost: larger, more capable models such as GPT-5.1 achieved higher and more consistent scores, whereas smaller, cheaper models performed worse, with lower overall accuracy. Regarding RQ2, zero-shot prompting proved more effective than role-based and chain-of-thought strategies in our experiments, suggesting that simpler instructions can outperform more elaborate prompts. Regarding RQ3, temperature had a measurable impact on model performance: GPT-5.1 remained largely stable across tested temperature values, while Devstral 2 2512 showed greater sensitivity, with its F1 score dropping noticeably at higher temperatures. Both models performed worse at temperature 1.5, indicating that excessive randomness is detrimental to classification reliability. However, this study is limited by the use of a single, relatively small dataset (CASTLE Benchmark), a focus on programs written in C, and the potential influence of the models' training data. Therefore, future research should explore larger, more diverse datasets, extend the analysis to additional programming languages, and investigate hybrid prompting strategies to further improve performance and real-world applicability.

REFERENCES

[1] World Bank, *Digital Progress and Trends Report 2023*, 2024. [Online]. Available: http://hdl.handle.net/10986/40970

[2] S. Kraus, P. Jones, N. Kailer, A. Weinmann, N. Chaparro-Banegas, and N. Roig-Tierno, "Digital Transformation: An Overview of the Current State of the Art of Research," *SAGE Open*, vol. 11, 2021, doi: 10.1177/21582440211047576.

[3] S. S. Goswami, S. Sarkar, K. K. Gupta, and S. Mondal, "The role of cyber security in advancing sustainable digitalization: Opportunities and challenges," *J. Decision Analytics and Intelligent Computing*, vol. 3, no. 1, pp. 270–285, 2023, doi: 10.31181/jdaic10018122023g.

[4] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights," arXiv preprint arXiv:2502.07049, 2025.

[5] K. Goseva-Popstojanova and A. Perhinschi, "On the capability of static code analysis to detect security vulnerabilities," *Inf. and Softw. Technol.*, 2015, doi: 10.1016/j.infsof.2015.08.002.

[6] G. Destefanis, S. Bartolucci, and M. Ortu, "A preliminary analysis on the code generation capabilities of GPT-3.5 and Bard AI models for Java functions," arXiv preprint arXiv:2305.09402, 2023.

[7] I. Priescu, G. S. Banu, T. C. Dosescu, and M. I. Banu, "Prompt Engineering in Cybersecurity – Achieving Technological Edge," 2025. [Online]. Available: https://reference-global.com/download/article/10.2478/raft-2025-0028.pdf

[8] R. A. Dubniczky *et al.*, "CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection," arXiv preprint arXiv:2503.09433, 2025.

[9] O. Genc, "From Zero-Shot to Role-Aware Reasoning: Exploring Prompt Strategies for AI-Assisted Industrial Symbiosis Identification," 2025.